Métodos de Optimización

Clase 3: Metaheurísticas

[Nombre del profesor]

Septiembre de 2026

Programa de Maestría, [Nombre de la universidad / programa]

Contenido

1. Motivación: de random walk a las metaheurísticas
2. Basin hopping
3. El criterio de Metrópolis y la temperatura T
4. La magnitud del salto: `step_size`
5. Simulated annealing
6. Algoritmos genéticos
7. Particle swarm optimization
8. Síntesis: 9 métodos, un solo paisaje
9. Roadmap del curso
10. Ejercicios y referencias

Motivación

De la Clase 2 a las metaheurísticas

Lo que vimos en la Clase 2

Bisección y áurea (1D), grid search ($O(N^d)$), random walk y Nelder–Mead. Todos son **locales** o de **costo exponencial**: ninguno tiene una estrategia inteligente para escapar de un mínimo local malo.

La pregunta que quedó abierta

¿Cómo combinar la **eficiencia** de un optimizador local (como Nelder–Mead o el descenso de gradiente) con la **capacidad de escapar** de mínimos locales (como random walk, pero de forma más inteligente)?

Hoy: cuatro respuestas a esa pregunta, cada una con una idea distinta de cómo *inyectar aleatoriedad útil* en la búsqueda.

El paisaje de siempre

Seguimos con la misma función de 6 pozos de la Clase 2 (`scripts/_landscape.py`), donde ya vimos que random walk greedy se queda atrapado en $f = -0{,}54$ y hasta Powell (scipy) converge al falso mínimo $(0, 0)$ con $f \approx 0$. El mínimo global real está en $x^* \approx (-6{,}37,\ 3{,}92)$, $f^* \approx -3{,}29$.

Idea unificadora: random walk como caso límite

- Random walk **greedy** $=$ aceptar un salto peor con probabilidad 0 (nunca).
- Random walk **puro** $=$ aceptar un salto peor con probabilidad 1 (siempre).

Las metaheurísticas de hoy generalizan esto: aceptan un salto peor con una probabilidad **intermedia**, controlada por un parámetro de *temperatura* T.

Basin hopping

Basin hopping: la idea (Wales & Doye, 1997)

Algoritmo

1. Partir de $\mathbf{x}_{\text{cur}}$, un mínimo local (via descenso de gradiente u otro optimizador local).
2. Perturbar: $\mathbf{x}_{\text{prop}} = \mathbf{x}_{\text{cur}} + \Delta$, con Δ un salto aleatorio de magnitud `step_size`.
3. **Reoptimizar localmente** desde $\mathbf{x}_{\text{prop}} \to$ nuevo mínimo local $\mathbf{x}_{\text{new}}$, f_{new}.
4. **Aceptar** con el criterio de Metrópolis (siguiente sección): siempre si $f_{\text{new}} < f_{\text{cur}}$; si no, con probabilidad $e^{-\Delta E/T}$.
5. Guardar aparte el mejor $(\mathbf{x}_{\text{best}}, f_{\text{best}})$ visto en **toda** la corrida (no solo el actual).

La clave

Cada salto aterriza **en el fondo de una cuenca** (no en cualquier punto) porque siempre se reoptimiza — así *saltamos entre mínimos locales* en vez de caminar sin rumbo como random walk.

Basin hopping: código manual (núcleo)

```
def local_search(x0, lr=0.1, max_iter=2000, tol=1e-8):
    x = np.array(x0, dtype=float)
    for _ in range(max_iter):
        g = grad_f(*x)                      # gradiente numerico
        if np.linalg.norm(g) < tol: break
        x = x - lr*g                        # paso FIJO y chico:
    return x, float(f(*x))                  # evita "saltar" de cuenca

def basin_hopping_manual(x0, n_iter=40, step_size=7.0, T=1.5, seed=0):
    rng = np.random.default_rng(seed)
    x_cur, f_cur = local_search(x0)
    x_best, f_best = x_cur.copy(), f_cur
    for it in range(n_iter):
        x_prop = x_cur + rng.uniform(-step_size, step_size, size=2)
        x_new, f_new = local_search(x_prop)
        aceptado = (f_new < f_cur) or \
                   (rng.random() < np.exp(-(f_new - f_cur)/T))
        if aceptado: x_cur, f_cur = x_new, f_new
        if f_new < f_best: x_best, f_best = x_new.copy(), f_new
    return x_best, f_best
```

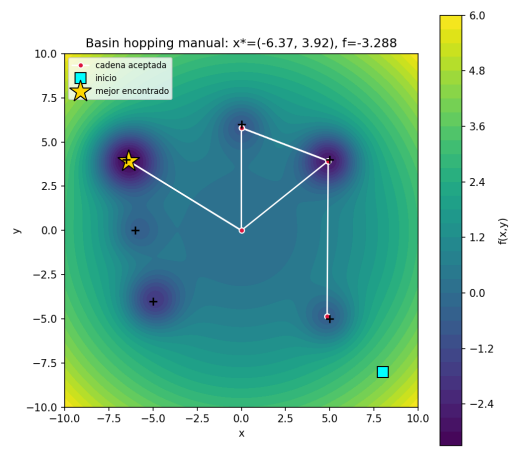
Basin hopping: ejemplo paso a paso

Arrancando en $x_0 = (8, -8)$, `step_size=7.0`, $T = 1{,}5$:

```
it= 1  x_new=[ 4.85 -4.85]  f_new=-0.5442  f_cur=-0.5442  -> ACEPTADO
it= 2  x_new=[-0.  -0. ]  f_new=-0.0000  f_cur=-0.5442  -> rechazado
it= 4  x_new=[4.91  3.93]  f_new=-2.7920  f_cur=-0.5442  -> ACEPTADO
it= 6  x_new=[0.    5.80]  f_new=-0.7552  f_cur=-2.7920  -> ACEPTADO (peor, sorteo!)
it= 9  x_new=[-6.37  3.92]  f_new=-3.2880  f_cur=-0.0000  -> ACEPTADO
      ...
it=40  ->  x* = [-6.3698, 3.9191]   f(x*) = -3.2880   (el GLOBAL)
```

En la iteración 6 se acepta un punto **peor** que el actual — eso es lo que permite, más adelante, escapar hacia el pozo global en la iteración 9. Sin ese *sorteo* el algoritmo se habría quedado en $(4{,}91, 3{,}93)$, $f = -2{,}79$.

Basin hopping: trayectoria completa



Basin hopping: manual vs. scipy

Con scipy (mismos parámetros):

```
from scipy.optimize import basinhopping

res = basinhopping(
    lambda p: f(p[0], p[1]), x0,
    niter=40,
    minimizer_kwargs={"method": "BFGS"},
    stepsize=7.0, T=1.5, seed=0)
```

Manual: $x^* = (-6{,}37, 3{,}92)$, $f^* = -3{,}288$ (**el global**).

Scipy: $x^* = (4{,}91, 3{,}93)$, $f^* = -2{,}792$ (un local, **no** el global).

Lección: metaheurística $\neq$ garantía

Ambas implementaciones son razonables; la diferencia viene de la **secuencia de aleatoriedad** y del optimizador local usado (gradiente fijo vs. BFGS). Basin hopping mejora las probabilidades de encontrar el global, **no las garantiza** en una sola corrida. Con más iteraciones, ambas convergerían con alta probabilidad.

Script completo: `scripts/01_basin_hopping.py`

Metrópolis y temperatura

El criterio de Metrópolis

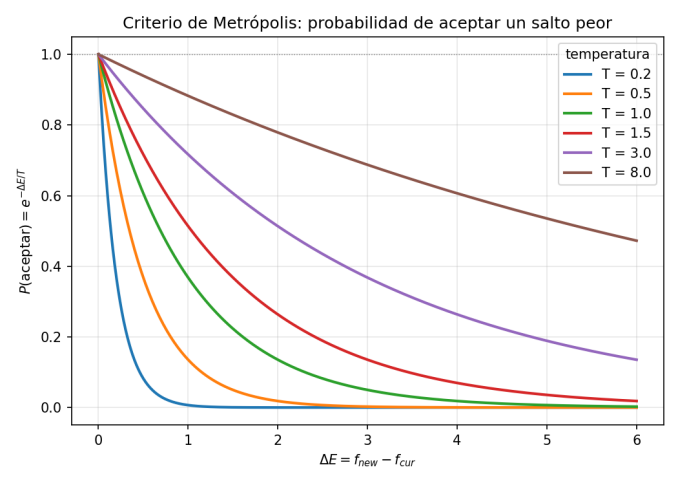
Regla de aceptación

Sea $\Delta E = f_{\text{new}} - f_{\text{cur}}$ (positivo si el salto *empeora*).

$$P(\text{aceptar}) = \begin{cases} 1 & \text{si } \Delta E \leq 0 \text{ (mejora o queda igual)} \\ e^{-\Delta E/T} & \text{si } \Delta E > 0 \text{ (empeora)} \end{cases}$$

- $T \to 0$: $e^{-\Delta E/T} \to 0$ para cualquier $\Delta E > 0 \;\Rightarrow$ **nunca** acepta empeorar = random walk *greedy*.
- $T \to \infty$: $e^{-\Delta E/T} \to 1 \;\Rightarrow$ **siempre** acepta = random walk *puro*.
- T intermedio: acepta empeorar **a veces**, más seguido si ΔE es chico (un empeoramiento leve) que si es grande.

Curvas $P(\text{aceptar})$ vs. ΔE, por temperatura



El sorteo: cómo se implementa una probabilidad

La idea

Para aceptar con probabilidad $P \in [0, 1]$: sortear $u \sim \mathcal{U}(0, 1)$ y aceptar si $u < P$. Como u es uniforme, $\Pr(u < P) = P$ exactamente.

```
aceptado = (f_new < f_cur) or (rng.random() < np.exp(-dE/T))
```

Verificación empírica (100,000 sorteos por caso):

DeltaE	T	P teorica	tasa observada
1.00	0.50	0.1353	0.1376
2.50	1.50	0.1889	0.1885
0.30	1.00	0.7408	0.7405
4.00	2.00	0.1353	0.1352

La tasa observada coincide con P hasta el tercer decimal: el sorteo hace exactamente lo que promete. Script: `scripts/02_metropolis_y_temperatura.py`

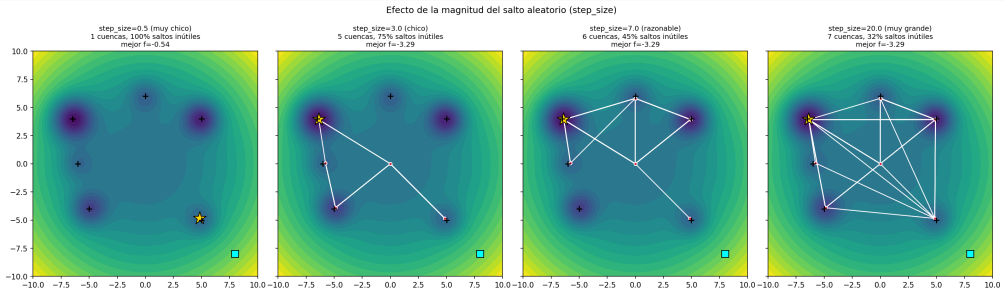
Magnitud del salto: step_size

¿Qué tan grande debe ser el salto aleatorio?

El compromiso

- **step_size muy chico**: el salto casi nunca saca al punto de la cuenca actual ⇒ `local_search` vuelve casi siempre al mismo mínimo ⇒ se queda atascado (basin hopping degenera a optimización local pura).
- **step_size muy grande**: el salto cae en zonas *lejanas y casi al azar* del paisaje ⇒ se pierde la ventaja de *caminar entre cuencas vecinas* y el método degenera a **random restart** puro (equivalente a probar puntos iniciales al azar, sin aprovechar la estructura local).
- **step_size razonable**: comparable a la **distancia típica entre mínimos locales** del paisaje.

Efecto del `step_size`: experimento



`step_size=0.5`: 100 % de los saltos son *inútiles* (vuelven a la misma cuenca), 1 sola cuenca visitada, no encuentra el global. `step_size=7.0` (comparable a la separación entre pozos, $\sim$ 9–13 unidades): mejor balance, visita 6 cuencas y encuentra el global. Script: `scripts/03_step_size_efecto.py`

Simulated annealing

Simulated annealing: la idea (Kirkpatrick et al., 1983)

Diferencia clave con basin hopping

Basin hopping perturba y **siempre reoptimiza localmente** antes de comparar. Simulated annealing perturba el punto actual y **compara/acepta directamente** con Metrópolis, **sin** minimizar en cada paso. El *aterrizaje* en un mínimo ocurre solo al final, cuando T ya es chica.

Esquema de enfriamiento (templado)

T empieza alta (explora como random walk puro) y decrece con las iteraciones (converge como random walk greedy). El más simple y usado es el **geométrico**:

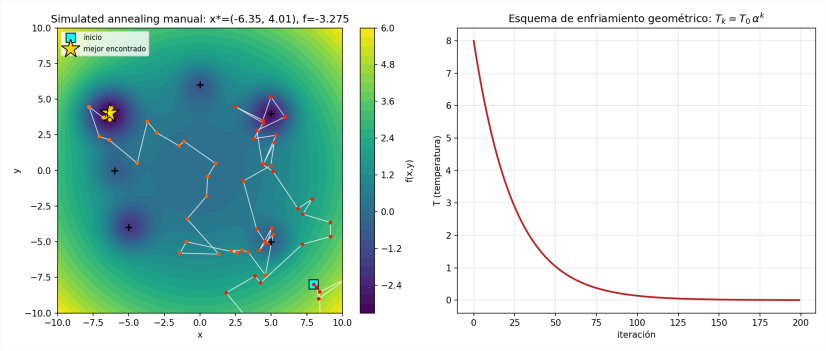
$$T_k = T_0\,\alpha^k, \qquad 0 < \alpha < 1 \text{ (típicamente 0,90–0,99)}$$

Así, simulated annealing es literalmente el *puente* entre random walk greedy y puro que anticipamos en la Clase 2, con T variando en el tiempo en vez de fija.

Simulated annealing: código manual

```
def simulated_annealing_manual(x0, T0=8.0, alpha=0.96, n_iter=200,
                               paso=1.5, seed=0):
    rng = np.random.default_rng(seed)
    x_cur = np.array(x0, dtype=float);  f_cur = float(f(*x_cur))
    x_best, f_best = x_cur.copy(), f_cur
    T = T0
    for it in range(n_iter):
        x_prop = x_cur + rng.normal(scale=paso, size=2)   # SIN reoptimizar
        f_prop = float(f(*x_prop))
        dE = f_prop - f_cur
        aceptado = dE < 0 or rng.random() < np.exp(-dE / T)
        if aceptado: x_cur, f_cur = x_prop, f_prop
        if f_cur < f_best: x_best, f_best = x_cur.copy(), f_cur
        T *= alpha                                        # enfriar
    return x_best, f_best
```

Simulated annealing: trayectoria y enfriamiento



Izquierda: la cadena difunde ampliamente al inicio (T alta) y se estabiliza cerca del mejor pozo al final ($T \approx 0$). Derecha: enfriamiento geométrico $T_k = T_0\alpha^k$. Manual: $f^* = -3{,}275$; `scipy.optimize.dual_annealing`: $f^* = -3{,}288$ (esquema de enfriamiento y reinicios más sofisticados). Script: `scripts/04_simulated_annealing.py`

Algoritmos genéticos

Algoritmos genéticos: la idea (Holland, 1975)

De un punto a una población

En vez de mover un solo punto, mantenemos N soluciones candidatas a la vez y las hacemos *evolucionar*:

1. **Fitness**: qué tan buena es cada solución ($-f$, si minimizamos).
2. **Selección**: los mejores individuos tienen más chance de *reproducirse* (aquí: torneo — se sortean k individuos y gana el mejor).
3. **Cruce (crossover)**: combinar dos padres para crear un hijo. Con valores reales, cruce **aritmético**: hijo $= \beta\, p_1 + (1-\beta)\, p_2$, $\beta \sim U(0,1)$.
4. **Mutación**: perturbación aleatoria chica, con probabilidad p_{mut}, para mantener diversidad.
5. **Elitismo**: los mejores individuos pasan sin cambios a la siguiente generación (no se pierde lo mejor logrado).

Algoritmos genéticos: código manual (núcleo)

```
def algoritmo_genetico(n_pob=30, n_gen=60, k_torneo=3, p_mut=0.25,
                       sigma_mut=1.0, n_elite=2, seed=0):
    pob = poblacion_inicial(n_pob, rng)          # N puntos al azar
    fit = -f(pob[:,0], pob[:,1])
    for gen in range(n_gen):
        orden = np.argsort(-fit);  pob, fit = pob[orden], fit[orden]
        nueva_pob = [pob[i].copy() for i in range(n_elite)]  # elitismo
        while len(nueva_pob) < n_pob:
            p1 = seleccion_torneo(pob, fit, k_torneo, rng)
            p2 = seleccion_torneo(pob, fit, k_torneo, rng)
            hijo = cruce_aritmetico(p1, p2, rng)
            hijo = mutar(hijo, p_mut, sigma_mut, rng)
            nueva_pob.append(hijo)
        pob = np.array(nueva_pob);  fit = -f(pob[:,0], pob[:,1])
```

La **diversidad de la población** juega el papel que en basin hopping/SA jugaba el salto aleatorio: varios individuos exploran distintas cuencas **en paralelo**.

Selección por torneo: la idea, sin código

Dos pasos, nada más

1. **Sorteo**: de *toda* la población (los N individuos completos, no un subconjunto ya filtrado), se sacan k al azar, con la misma probabilidad para todos sin importar su fitness.
2. **Torneo**: de esos k sorteados, gana el de mejor fitness. Sin más azar – es una comparación directa.

Un error común

Torneo **no** empieza escogiendo de antemano "los mejores" y sorteando ahí adentro – eso sería *truncamiento*, donde los débiles quedan eliminados de raíz. En torneo el sorteo siempre parte de la población completa; el filtro hacia lo bueno ocurre indirectamente, sorteo tras sorteo, no de una vez.

El tamaño k del sorteo controla la presión selectiva: k grande $\Rightarrow$ casi siempre gana el mejor de todos (muy elitista, poca diversidad); k chico $\Rightarrow$ más fácil que gane alguien mediocre por competir solo contra peores (más diversidad).

Dos padres, dos sorteos – y la élite también compite

- Cada hijo necesita dos padres (p_1, p_2): son **dos sorteos independientes**, cada uno con su propio grupo de k individuos extraído de la población completa. Como cada sorteo vuelve a empezar de cero, p_1 y p_2 pueden terminar siendo el mismo individuo – no hay ninguna regla que lo impida.
- El **elitismo** (los n_{elite} mejores pasan intactos a la siguiente generación) y la **selección por torneo** son dos mecanismos distintos que conviven: la élite **no se excluye** del sorteo – sigue en la población y, gracias a su buen fitness, tiene muy buenas chances de además ganar un torneo y convertirse en padre.
- La diferencia real entre ser élite o no: la élite tiene *asegurada* una copia en la siguiente generación sin pasar por el sorteo; el resto solo puede influir en la siguiente generación si gana un torneo.

Cruce aritmético: ¿por qué β es aleatorio?

```python
def cruce_aritmetico(p1, p2, rng):
    beta = rng.random()          # beta ~ U(0,1), nuevo en CADA cruce
    return beta * p1 + (1 - beta) * p2
```

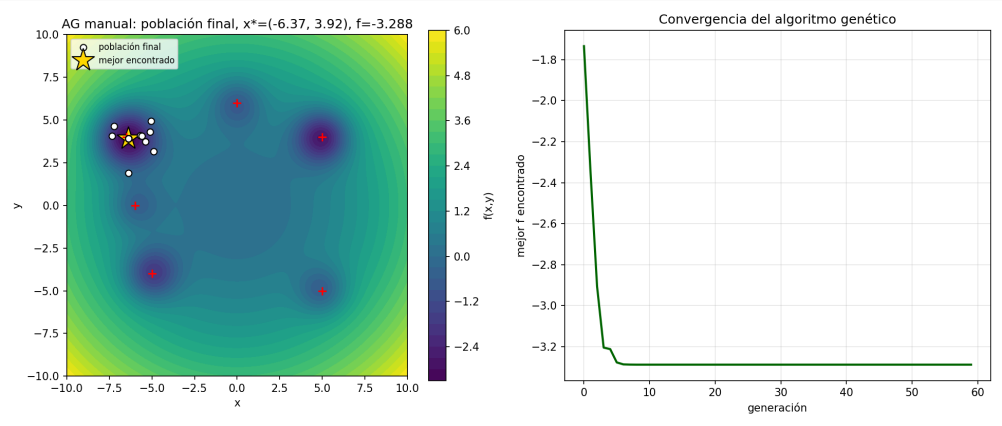
β decide en qué punto del segmento que une a los padres cae el hijo: β cercano a 1 $\rightarrow$ hijo casi igual a p_1; cercano a 0 $\rightarrow$ casi igual a p_2; $\beta = 0{,}5$ $\rightarrow$ exactamente el punto medio.

¿Por qué no fijar $\beta = 0{,}5$ siempre?

Si todo hijo fuera siempre el promedio exacto de sus padres, la población tendería a colapsar hacia el centro del espacio de búsqueda muy rápido, perdiendo diversidad. Sortear β en cada cruce hace que unos hijos se parezcan más a un padre y otros al otro, conservando variedad.

Nota: aquí se sortea un único β por cruce (mismo valor para x e y); existe una variante con un β distinto por coordenada.

Algoritmos genéticos: población final y convergencia



30 individuos, 60 generaciones: converge al global en $\sim$10 generaciones y la población final se concentra alrededor del mejor pozo. $x^* = (-6{,}37, 3{,}92)$, $f^* = -3{,}288$.

Algoritmos genéticos: manual vs. scipy

```
from scipy.optimize import differential_evolution

res = differential_evolution(
    lambda p: f(p[0], p[1]),
    bounds=[(-10,10),(-10,10)],
    popsize=15, maxiter=60, seed=0)
```

`differential_evolution` es también un algoritmo evolutivo poblacional (misma familia), pero con un operador de mutación distinto (diferencias vectoriales entre individuos, en vez de gaussiana) — típicamente más robusto y rápido para uso general. Ambos llegan a $f^* = -3{,}288$ (el global). Script: `scripts/05_algoritmos_geneticos.py`

Particle swarm optimization

PSO: la idea (Kennedy & Eberhart, 1995)

Un enjambre que *vuela* sobre el paisaje

Cada partícula i tiene posición $\mathbf{x}_i$ y velocidad $\mathbf{v}_i$, y recuerda su mejor posición histórica $\mathbf{p}_i$ (*personal best*). El enjambre completo conoce la mejor posición global $\mathbf{g}$ (*global best*).

$$\mathbf{v}_i \leftarrow \underbrace{w\,\mathbf{v}_i}_{\text{inercia}} + \underbrace{c_1 r_1 (\mathbf{p}_i - \mathbf{x}_i)}_{\text{cognitivo}} + \underbrace{c_2 r_2 (\mathbf{g} - \mathbf{x}_i)}_{\text{social}} \qquad \mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$$

con $r_1, r_2 \sim U(0,1)$ (aleatoriedad en *cuánto* confiar en cada tendencia, no en si moverse o no).

- w grande $\rightarrow$ más exploración; chico $\rightarrow$ converge más rápido.
- c_1: qué tanto confía la partícula en su **propia** experiencia.
- c_2: qué tanto confía en la experiencia del **grupo**.

PSO: código manual (núcleo)

```
def pso_manual(n_particulas=25, n_iter=60, w=0.7, c1=1.5, c2=1.5, seed=0):
    x = rng.uniform(lo, hi, size=(n_particulas, 2))
    v = rng.uniform(-1, 1, size=(n_particulas, 2))
    p_best = x.copy();  f_p_best = f(x[:,0], x[:,1])
    g_best = p_best[np.argmin(f_p_best)].copy()
    for it in range(n_iter):
        r1, r2 = rng.random((n_particulas,2)), rng.random((n_particulas,2))
        v = w*v + c1*r1*(p_best - x) + c2*r2*(g_best - x)
        x = np.clip(x + v, lo, hi)
        f_x = f(x[:,0], x[:,1])
        mejora = f_x < f_p_best
        p_best[mejora], f_p_best[mejora] = x[mejora], f_x[mejora]
        ig = np.argmin(f_p_best)
        if f_p_best[ig] < f_g_best: g_best, f_g_best = p_best[ig].copy(), f_p_best[ig]
```

PSO: ¿cómo arrancan `p_best` y `g_best`?

`p_best` inicial: la posición de nacimiento

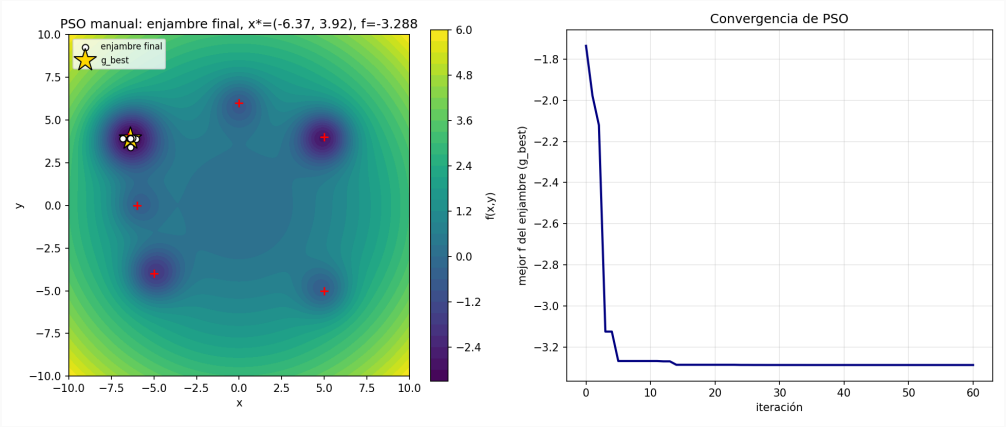
Cada partícula solo conoce, al inicio, el punto aleatorio donde nació – no hay nada más con qué comparar todavía. Por eso `p_best = x.copy()`: su memoria personal arranca siendo, simplemente, su propio punto de partida.

`g_best` inicial: la mejor entre las N posiciones de nacimiento

Aquí sí hay una comparación real: se evalúa f en las N posiciones iniciales y se toma la que dio el valor más bajo. No es al azar – es una búsqueda directa del mínimo entre N valores ya calculados.

Al momento cero cada partícula "cree" que su propio punto de partida es lo mejor que existe (es lo único que conoce), pero el enjambre ya sabe cuál de esas N creencias individuales es, objetivamente, la mejor – y eso le da una dirección común desde la primera iteración, en vez de volar sin rumbo.

PSO: enjambre final y convergencia



25 partículas, 60 iteraciones: el enjambre converge y se agrupa alrededor del mínimo global en $\sim$15 iteraciones — visiblemente más rápido que GA en este paisaje. $x^* = (-6{,}37, 3{,}92)$, $f^* = -3{,}288$.

PSO: ¿y la comparación con scipy?

Scipy no tiene PSO nativo

`scipy.optimize` no incluye particle swarm optimization. La familia evolutiva/poblacional más cercana disponible sigue siendo `differential_evolution` (mismo espíritu: población, sin gradiente, sin garantías de optimalidad global, pero robusto en la práctica).

- Para PSO *de verdad* en Python, la librería externa de referencia es **`pyswarms`** (no incluida en scipy).
- Vale la pena notarlo como ejemplo de que **no todos los métodos clásicos de optimización global tienen una función lista en scipy** — a veces hay que implementarlos, o buscar una librería especializada.

Script completo: `scripts/06_particle_swarm.py`

Síntesis

Nueve métodos, un mismo paisaje

	Grid search	Random walk	Nelder–Mead	Basin hopping	Sim. annealing
Tipo	exhaustivo	estocástico	local	metaheur.	metaheur.
Población	–	1 punto	1 simplex	1 cadena	1 cadena
Escapa mínimos locales	sí (fuerza bruta)	no	no	sí	sí
Costo	$O(N^d)$	$O(\text{iter})$	$O(\text{iter})$	$O(\text{iter} \times \text{local})$	$O(\text{iter})$

	Algoritmos genéticos	PSO
Tipo	metaheur. poblacional	metaheur. poblacional
Población	N individuos	N partículas
Escapa mínimos locales	sí (diversidad)	sí (diversidad)
Costo	$O(\text{iter} \times N)$	$O(\text{iter} \times N)$

Los tres últimos (basin hopping, SA, GA/PSO) todos usan aleatoriedad **con criterio**: no exploran a ciegas como random walk puro, ni exhaustivamente como grid search.

Lo que tienen en común

El hilo conductor de las Clases 2 y 3

Todas las metaheurísticas de hoy resuelven el mismo problema: **cómo decidir cuándo aceptar una solución peor** para poder escapar de mínimos locales, sin degenerar en búsqueda puramente aleatoria.

- Basin hopping y SA: un parámetro de temperatura T (fijo o decreciente) controla esa probabilidad, vía Metrópolis.
- GA y PSO: la **diversidad de una población** cumple un papel similar — mientras algunos individuos convergen, otros siguen explorando.

Ninguno garantiza el óptimo global

Todos son heurísticos: mejoran mucho la probabilidad de encontrar el mínimo global frente a un método puramente local, pero **ninguno lo garantiza** en tiempo finito (salvo grid search, que sí lo garantiza pero a costo exponencial).

Un detalle que comparten los cuatro: ¿cuándo paran?

En los scripts de esta clase

Basin hopping, SA, GA y PSO se detienen aquí por **presupuesto fijo**: un número prefijado de iteraciones o generaciones (`n_iter`, `n_gen`), sin preguntar en ningún momento si ya conviene parar. Es el criterio más fácil de programar, pero también el menos eficiente.

Lo que se usa en la práctica

- **Estancamiento**: parar si el mejor valor no mejora en las últimas M iteraciones/generaciones seguidas.
- **Meta de calidad conocida**: parar al alcanzar un valor objetivo – solo sirve si se conoce de antemano, como en un ejemplo de clase; en un problema real casi nunca se tiene.
- **Diversidad/dispersión**: parar si la población o el enjambre ya se agruparon mucho (poca varianza entre ellos).
- **Presupuesto de cómputo real**: tiempo de reloj o número de evaluaciones de f, útil cuando evaluar f es caro (por ejemplo, si cada evaluación corre una simulación).

Roadmap del curso

¿Qué sigue después de las Clases 1–3?

Ya cubrimos

- Clase 1: fundamentos (gradiente, Hessiana, convexidad, Lagrange/KKT).
- Clases 2–3: métodos de búsqueda, de lo determinista (bisección, grid search) a lo metaheurístico (basin hopping, SA, GA, PSO).

Lo que falta

- Método de **Newton** y métodos **cuasi-Newton** (BFGS, L-BFGS): usar curvatura (Hessiana o su aproximación) para converger mucho más rápido que el gradiente simple.
- **Programación lineal**: simplex, punto interior.
- **Programación no lineal con restricciones**: SQP, penalizaciones, barreras.

Ejercicios propuestos

Ejercicios para la próxima clase

1. Modifique `scripts/01_basin_hopping.py` probando `step_size` $\in \{2, 5, 10, 15\}$ y $T \in \{0{,}3, 1{,}5, 5\}$. ¿Qué combinación encuentra el global con más frecuencia en 10 corridas con semillas distintas?
2. Implemente un esquema de enfriamiento **lineal** ($T_k = T_0(1 - k/n_{\text{iter}})$) para simulated annealing y compárelo contra el geométrico del script `04_simulated_annealing.py`.
3. En el algoritmo genético, quite el elitismo (`n_elite=0`) y observe qué pasa con la curva de convergencia. ¿Por qué puede el mejor `f` *empeorar* de una generación a otra sin elitismo?
4. Modifique PSO para que w decrezca linealmente de 0,9 a 0,4 durante las iteraciones (*inertia weight damping*, muy usado en la práctica). ¿Mejora la convergencia?
5. **Comparación final:** corra los 4 métodos de esta clase 10 veces cada uno (semillas distintas) sobre el paisaje de 6 pozos, y grafique un histograma de f^* encontrado por cada método. ¿Cuál es más consistente?
6. **Reflexión:** explique con sus palabras por qué basin hopping y GA/PSO pueden considerarse *primos*: ambos mantienen múltiples candidatos (una cadena con reinicios vs. una población), aunque basin hopping los explore uno a la vez.

Referencias

Referencias

- D. Wales, J. Doye. *Global Optimization by Basin-Hopping and the Lowest Energy Structures of Lennard-Jones Clusters*. J. Phys. Chem. A, 101(28), 1997.
- S. Kirkpatrick, C. Gelatt, M. Vecchi. *Optimization by Simulated Annealing*. Science, 220(4598), 1983.
- J. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.
- D. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.
- J. Kennedy, R. Eberhart. *Particle Swarm Optimization. Proc. IEEE Int. Conf. on Neural Networks*, 1995.
- M. Kochenderfer, T. Wheeler. *Algorithms for Optimization*. MIT Press, 2019. (Cap. 8–9: metaheurísticas poblacionales y basadas en temperatura, con código.)
- J. Nocedal, S. Wright. *Numerical Optimization*, 2.ª ed. Springer, 2006.
- Documentación de `SciPy` (`scipy.optimize`): `https://scipy.org`.
- `pyswarms`: librería especializada de PSO en Python (no incluida en scipy): `https://pyswarms.readthedocs.io`.

¿Preguntas?